Lossless compression (III)
Dictionary techniques, Context-based compression …

Karel Fliegel
(fliegek@fel.cvut.cz)

MultiMedia Technology Group

Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Radioelectronics

Technická 2
166 27 Prague 6
Czech Republic

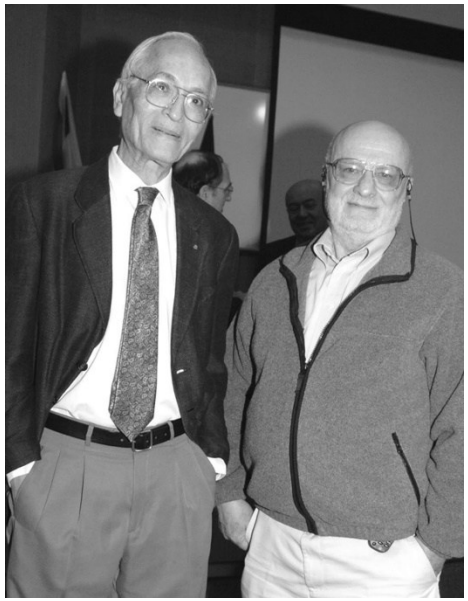
13/3/2026

© Multimedia Technology Group

Contents

- **Dictionary techniques**
  - **Introduction**
  - **Static** dictionary
  - **Adaptive** dictionary (**LZ77**, **LZ78**, **LZW**)
  - **Applications** (**GIF**, **PNG**)

- **Context based compression**
  - **Introduction**
  - **Prediction** with **partial match** (**ppm**)
  - **Burrows-Wheeler** Transform (**BWT**)
  - **Move-to-Front** (**mtf**) coding
  - **Run-Length** (**En**)**coding** (**RLE**)



Jacob Ziv (1931 - 2023)
Abraham Lempel (1936 - 2023)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: Introduction

- **Source generating sequence** of **independent symbols**
  - **Efficient** application of **variable length coding** (**VLC**)
    - **Suitable** for **Huffman** or **Arithmetic coding**
    - Exploiting **source statistics** (**more probable/less probable** symbols)
  - For **correlated sources** → **decorrelation step** needed
- **Dictionary techniques**
  - Incorporate **structure** in **data** → **increasing amount** of **compression**
  - Building **list** of **commonly occurring patterns**
    - **Static** or **dynamic** (**adaptive**) **dictionary** design
    - Pattern **encoding** using **list indices**
    - Useful for **sources** generating **small number** of **frequent patterns**
  - **Typical applications**
    - **Text compression, image compression, …**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Dictionary techniques: Introduction

- **Sources** with **reoccurring patterns**
  - Classic **text source** example → **words** reoccur **constantly** → **some patterns** do **not occur** or with **rarity**
  - Reasonable **coding approach** → **keeping** list (**dictionary**) of **frequently occurring patterns**
    - **Pattern appears** in **source output** → **encoded** as **reference** to **dictionary**
    - **Pattern not** in **dictionary appears** → **less efficient coding** used
    - **Input split** into **two classes**
      - (**1**) **Frequently occurring** patterns
        - **Effective** encoding → **dictionary much smaller** than **number** of **all patterns**
      - (**2**) **Infrequently occurring** patterns
  - **Example**
    - Having **particular text** with **four-character words**, while having characters **lowercase** plus **punctuation mark**
      - Suppose 26 **lowercase letters** and 6 **punctuation marks** (comma, period, exclamation, question, semicolon, colon)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: Introduction

Sources with reoccurring patterns

Example

- Having **particular text** with **four-character words**, while having characters **lowercase** plus **punctuation mark**
  - Suppose 26 **lowercase letters** and 6 **punctuation marks** (comma, period, exclamation, question, semicolon, colon) → **alphabet size** 32
  - **One character** at **time** encoding → 5 **bits** per **character** ($2^5 = 32$)
  - **Four character pattern** (equally likely) → 20 **bits** per **pattern** ($2^{20} = 1\,048\,576$)
  - Putting 256 **most likely** (four-character) **patterns** into **dictionary**
  - **Transmission scheme**
    - **Sending pattern** in **dictionary** → 1bit **flag**, e.g. 0 → **followed** by 8bit **index**
    - **Sending pattern not** in **dictionary** → 1bit **flag**, e.g. 1 → **followed** by 20bit **encoding**
    - **More bits used** for **patterns not** in **dictionary** → 21 **instead** of 20bit → **only** 9bit for **dictionary ones**
    - **Utility** of **scheme** → **depends** on **percentage** of **words encountered** in **dictionary**
    - **Average number** of **bits per pattern** $R = 9p + 21(1-p) = 21 - 12p$ (**probability** of **dictionary pattern**)
    - R should be **less than** 20bit → $p > 0.0833$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: Introduction

- **Sources** with **reoccurring patterns**
  - **Efficient encoding** → p as **large** as **possible**
    - **Increasing probability** p of **pattern** in **dictionary**
    - Carefully **selecting patterns** most **likely** to **occur** for **dictionary**
  - **Acquiring information** on **pattern occurrence**
    - **Static** approach → having **sufficient prior knowledge**
    - **Adaptive** (**dynamic**) approach → **acquiring** knowledge **while encoding**
- **Static dictionary**
  - **Appropriate** for **prior knowledge** → **specific applications**
    - **Utilization** of **reoccurring patterns** → **application specific**
      - For example **database records** in **company employee database**
      - **Static dictionary scheme works** only for **application/data designed**
      - In **different application** → may even **cause data expansion** instead **compression**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Dictionary techniques: Static dictionary

Static dictionary

Digram coding

- **Dictionary** consists of **all alphabet letters** followed by pairs (**digrams**)
  - For example 95 **printable ACSII** followed by 161 **pairs** → 256 **dictionary**
  - 161 **pairs** (**digrams**) → **most frequently** used **pairs** of **characters**
- **Diagram encoder reads two-character input** and **searches dictionary**
  - If **in dictionary** → corresponding **index transmitted**
  - If **not in dictionary** → **first** character **encoded** → **second character** becomes **first character** of **next digram**

Example

- Source five-letter **alphabet** $\mathcal{A} = \{a, b, c, d, r\}$ with **prior knowledge**
  - Encoding **sequence** $a\ b\ r\ a\ c\ a\ d\ a\ b\ r\ a$
  - (**1**) **Sample dictionary** is **built** based on **prior knowledge**
  - (**2**) **Sequence** is **encoded**

$$a\ b, r, a\ c, a\ d, a\ b, r, a \rightarrow$$
$$101\ 100\ 110\ 111\ 101\ 100\ 000$$

Code	Entry	Code	Entry
000	a	100	r
001	b	101	ab
010	c	110	ac
011	d	111	ad

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: Static dictionary

Static dictionary

Digram coding

- **Dictionary designed** for **prior knowledge** → **specific applications**
- In **different application** → may even **cause data expansion** instead **compression**
  - Two **examples** for **English textbook** and **C programs**

Pair	Count	Pair	Count
e♯	1128	*ar*	314
♯t	838	*at*	313
♯♯ **Space**	823	*♯w*	309
th	817	*te*	296
he	712	*♯s*	295
in	512	*d♯*	272
s♯	494	*♯o*	266
er	433	*io*	257
♯a	425	*co*	256
t♯	401	*re*	247
en	392	*♯$*	246
on	385	*r♯*	239
n♯	353	*di*	230
ti	322	*ic*	229
♯i	317	*ct*	226

Thirty most frequently occurring pairs of characters in 41 364 characters long LaTeX document

Pair	Count	Pair	Count
♯♯	5728	*st*	442
nl♯	1471	*le*	440
;nl	1133	*ut*	440
in	985	*f(*	416
nt	739	*ar*	381
=♯	687	*or*	374
♯i	662	*r♯*	373
t♯	615	*en*	371
♯=	612	*er*	358
);	558	*ri*	357
,♯	554	*at*	352
nlnl **New line**	506	*pr*	351
♯f	505	*te*	349
e♯	500	*an*	348
♯∗	444	*lo*	347

Thirty most frequently occurring pairs of characters in a collection of C programs with 64 983 characters

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: Adaptive dictionary

Adaptive dictionary

- **Most adaptive techniques based** on **LZ(W) approach**
  - **Jacob Ziv** (1931-2023, Palestine)
  - **Abraham Lempel** (1936-2023, Lviv, Poland/Ukraine)
    - **L&Z both** at **Technion** – **Israel Institute** of **Technology**
  - **Terry Archer Welch** (1939-1988, USA)
    - **Massachusetts Institute of Technology** (MIT)
    - **University of Texas at Austin**
- Two **basic adaptive dictionary** approaches from **1970s**
  - **LZ77** (**LZ1**) presented in **1977** and
  - **LZ78** (**LZ2**) presented in **1978**
  - **Transposition** of **letters LZ↔ZL caused** by historical **accident**
  - **Numerous variations** of **LZ77** and **LZ78**

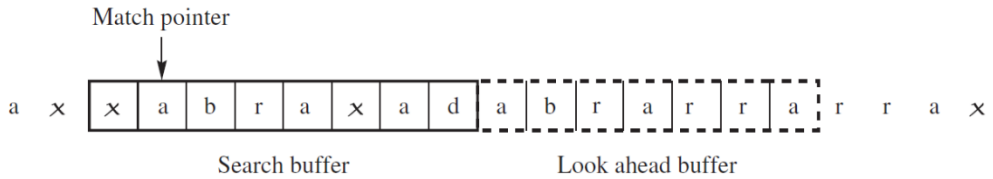
Abraham Lempel, **Jacob Ziv**, **Terry Welch**

Ziv, J., Lempel, A., A Universal Algorithm for Sequential Data Compression, IEEE Transactions on Information Theory, 1977.
Ziv, J., Lempel, A., Compression of Individual Sequences via Variable-Rate Coding, IEEE Transactions on Information Theory, 1978.
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ77

LZ77 approach

- **Dictionary** is **portion** of **previously encoded sequence**
  - **Encoder examines input sequence** through **sliding window**
  - **Window** consists of **two buffers**
    - **Search buffer** → contains **portion** of **recently encoded sequence**
    - **Lookahead buffer** → contains **next portion** of **sequence** to be **encoded**
- **Encoding process**
  - **Encoding sequence** in **lookahead buffer** → **encoder** moves **pointer** back **through search buffer**
    - (1) **Pointer moves back** until **first matching symbol** in **lookahead buffer**
    - (2) **Offset** (o) → **distance** of **pointer** from **lookahead buffer**
    - (3) **Length** of **match** (l) → **number** of **consecutive matching symbols** in **search buffer** → **encoder searches** for **longest match** in **search buffer**



Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ77

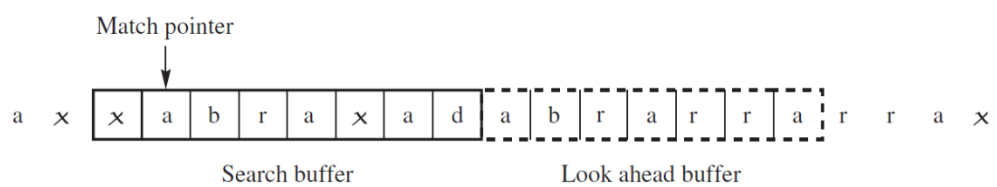
LZ77 approach

Encoding process

- **Encoding sequence** in **lookahead buffer** → **encoder** moves **pointer** back **through search buffer**
  - (1) **Pointer moves back** until **first matching symbol** in **lookahead buffer**
  - (2) **Offset** (o) → **distance** of **pointer** from **lookahead buffer**
  - (3) **Length** of **match** (l) → **number** of **consecutive matching symbols** in **search buffer** → **encoder searches** for **longest match** in **search buffer**
  - (4) **Once longest match found** → **encoded** with **triple** $\langle o, l, c \rangle$
    - **Offset** o, **length** of **match** l, and **codeword** c for **following symbol** in **lookahead** buffer

Example

- Match **pointer pointing** to **beginning of longest match**
  - **Offset** $o = 7$, **length** of **match** $l = 4$, and **following symbol** $c = r$ in **lookahead buffer**



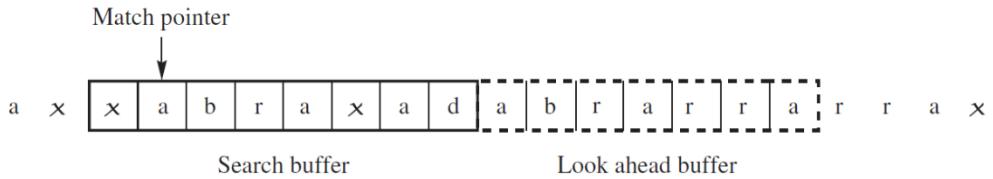
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ77

LZ77 approach

Encoding process

- **Encoding sequence** in **lookahead buffer** → **encoder** moves **pointer** back **through search buffer**
  - (1) **Pointer moves back** until **first matching symbol** in **lookahead buffer**
  - (2) **Offset** (o) → **distance** of **pointer** from **lookahead buffer**
  - (3) **Length** of **match** (l) → **number** of **consecutive matching symbols** in **search buffer** → **encoder searches** for **longest match** in **search buffer**
  - (4) **Once longest match found** → **encoded** with **triple** $\langle o, l, c \rangle$
    - **Offset** o, **length** of **match** l, and **codeword** c for **following symbol** in **lookahead** buffer
  - (5) For **no match** in **search buffer** → **triple** $\langle 0, 0, c \rangle$ → c **symbol itself**
    - Taking care of **situation** where **no match** for **symbol** in **lookahead** can be found in **search buffer**



Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ77

LZ77 approach

Encoding process

- **Encoding sequence** in **lookahead buffer** → **encoder** moves **pointer** back **through search buffer**
- **Parameters** for **LZ77**
  - **Offset** o, **length** of **match** l, and **codeword** c for **following symbol** in **lookahead**
  - **Size** of **search buffer** S
  - **Size** of **search** + **lookahead** buffers **window** W
  - Source **alphabet size** A
  - **Number** of **bits** to **code triple** $\langle o, l, c \rangle$ using **fixed-length code** $\lceil \log_2 S \rceil + \lceil \log_2 W \rceil + \lceil \log_2 A \rceil$
    - **Length** of **match** can **exceed length** of **search buffer** → $\lceil \log_2 W \rceil$
- **Three possibilities** in **encoding process**
  - (**1**) **There** is **no match** for **next character** to be **encoded** in **window**
  - (**2**) **There** is **match** for **next character** in **search buffer**
  - (**3**) **Matched** string **extends** to **lookahead buffer**

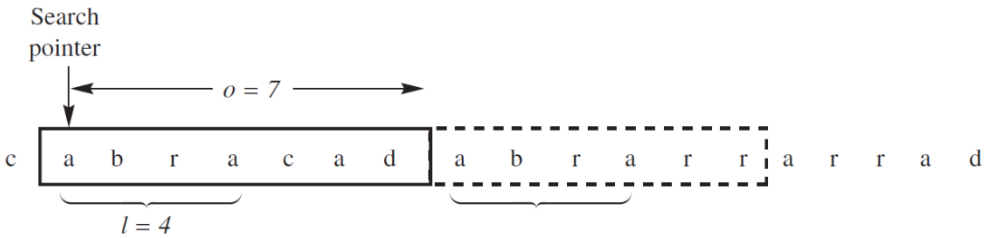
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ77

LZ77 approach

Example

- **Sequence** to be **encoded** … $c\ a\ b\ r\ a\ c\ a\ d\ a\ b\ r\ a\ r\ r\ a\ r\ r\ a\ d$ …
  - **Size** of **search buffer** $S = 7$
  - **Size** of **search** + **lookahead** buffers **window** $W = 13$
- **Encoding procedure**
  - **(1) Current** condition $(c\ a\ b\ r\ a\ c\ a)(d\ a\ b\ r\ a\ r)$ → **looking** for **match** for d → **no match triple** → $\langle o, l, c\rangle$ → $\langle 0, 0, C(d)\rangle$
  - **(2) Moving window** by **one character** $(a\ b\ r\ a\ c\ a\ d)(a\ b\ r\ a\ r\ r)$ → **looking** for **match** for a → **best match** $a\ b\ r\ a$ → **triple** $\langle o, l, c\rangle$ → $\langle 7, 4, C(r)\rangle$
  - **(3) Moving window** by **five characters** $(a\ d\ a\ b\ r\ a\ r)(r\ a\ r\ r\ a\ d)$ → **looking** for **match** for r → **best match** $r\ a\ r\ r\ a$ → **triple** $\langle o, l, c\rangle$ → $\langle 3, 5, C(d)\rangle$
  - **Encoded sequence after** $(c\ a\ b\ r\ a\ c\ a)$ → $\langle 0, 0, C(d)\rangle\langle 7, 4, C(r)\rangle\langle 3, 5, C(d)\rangle$



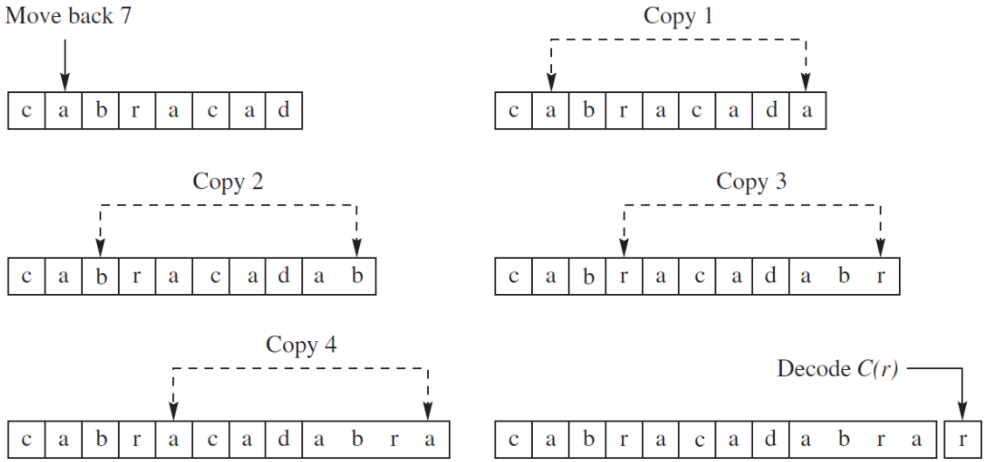
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ77

LZ77 approach

Example

- **Sequence** to be **encoded** … $c\ a\ b\ r\ a\ c\ a\ d\ a\ b\ r\ a\ r\ r\ a\ r\ r\ a\ d$ …
- **Decoding procedure**
  - **Decoded sequence** $(c\ a\ b\ r\ a\ c\ a)$ and **received triples** $\langle 0, 0, C(d)\rangle\langle 7, 4, C(r)\rangle\langle 3, 5, C(d)\rangle$
  - (**1**) First **triple** $\langle 0, 0, C(d)\rangle$ → **no match triple** → **decoded string** $c\ a\ b\ r\ a\ c\ a\ d$
  - (**2**) Second **triple** $\langle 7, 4, C(r)\rangle$ → **moving pointer** $o = 7$ **characters back** → copy $l = 4$ **characters** → **decoding** $C(r)$



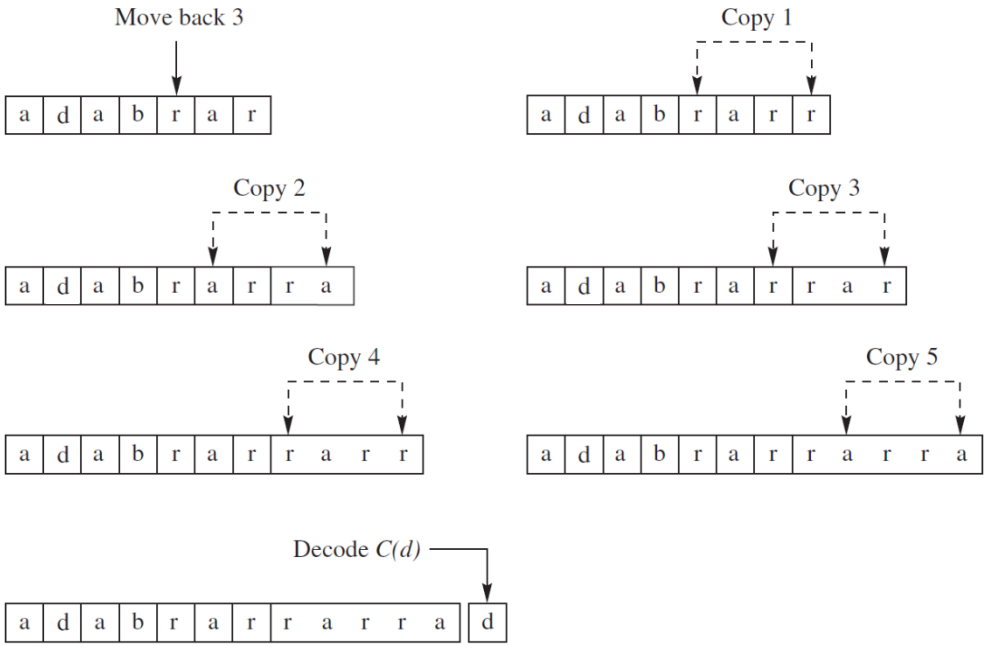
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ77

LZ77 approach

Example

- **Sequence** to be **encoded** … $c\ a\ b\ r\ a\ c\ a\ d\ a\ b\ r\ a\ r\ r\ a\ r\ r\ a\ d$ …
- **Decoding procedure**
  - **Received triples** $\langle 0, 0, C(d)\rangle\langle 7, 4, C(r)\rangle\langle 3, 5, C(d)\rangle$
  - (**3**) Third **triple** $\langle 3, 5, C(d)\rangle$ → **moving pointer** $o = 3$ **characters back** → copy $l = 5$ **characters** → **decoding** $C(d)$



Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ77

- **Variations** of **LZ77 approach**
  - **LZ77 scheme** → very **simple** → **no prior knowledge** required
  - **Jacob Ziv** and **Abraham Lempel showed**
    - **LZ77 asymptotically** approaches **performance** → **achievable** by **scheme** having **full knowledge** on **source statistics**
    - **Asymptotically** → there are **modifications improving LZ77** performance
  - **Improvements** mostly in **encoding triples** $\langle o, l, c \rangle$
    - **LZ77** → **triples encoded** using **fixed-length code**
    - **Accepting** more **complexity** → **triples encoded** using **variable-length code** (**VLC**)
      - **Adaptive VLC** or **semiadaptive** (two-pass) **VLC procedures**
      - **LZ77** followed by **VLC** → **popular compression packages PKZip**, **Zip**, **LHarc**, **PNG**, **gzip**, **ARJ**, …
    - **Another LZ77 variations**
      - **Variable size** of **search/lookahead buffers**
      - **Removing single character encoding**

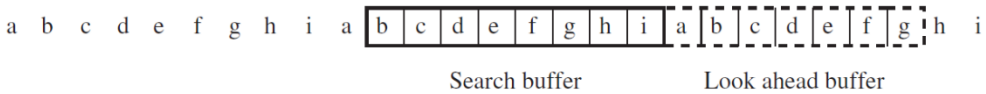
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Dictionary techniques: LZ78

LZ78 approach

- **LZ77 algorithm** assumes **like patterns** occurring **close** together
  - Utilizing **recent past** as **dictionary** for **encoding**
  - Patterns **recuring over longer period** than **window** → **not captured**
  - **Worst-case** scenario → **periodic sequence** and **period longer** than **buffer**



  - For example **periodic sequence** with **period length** 9 → **efficient encoding** for **search buffer** one **symbol longer**
    - **None** of **new symbols** matching **search buffer** → **represented** in separate **codewords**
    - Resulting in **expansion** rather than **compression**
- **LZ78 algorithm** solves **mentioned issue**
  - **Dropping reliance** on **search buffer** → **explicit dictionary** kept → to be built (in **identical** manner) in **encoder/decoder**
  - **Inputs encoded** in **doubles** $\langle i, c \rangle$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ78

LZ78 approach

- **Inputs encoded** in **doubles** $\langle i, c \rangle$
  - **Index** of **dictionary entry** (i) with **longest match** to **input**
  - **Code** for **character** (c) following **matched portion** of **input**
  - For **no match** $i = 0$ → becoming **newest dictionary entry**

- **Example**
  - **Sequence** to be **encoded** using **LZ78 approach** ($\not{b}$ stands for **space**)
    $wabba\not{b}wabba\not{b}wabba\not{b}wabba\not{b}woo\not{b}woo\not{b}woo$
  - **Encoding procedure**
    - (1) **Dictionary** is **empty** → first **sequence** of **doubles** $\langle i, c \rangle$ → $\langle 0, C(w) \rangle$ $\langle 0, C(a) \rangle$ $\langle 0, C(b) \rangle$
    - (2) **Fourth symbol** → **third entry** in **dictionary** → $\langle 3, C(a) \rangle$ → **adding pattern** ba as **fourth entry** in **dictionary**
    - **Continuing** in **same fashion** → **building dictionary**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZ78

LZ78 approach

Example

- **Sequence** to be **encoded** using **LZ78 approach** (*ƀ* stands for **space**)
  $wabbaƀwabbaƀwabbaƀwabbaƀwooƀwooƀwoo$
- **Encoding procedure**
  - **(1) Dictionary** is **empty** → first **sequence** of **doubles** $\langle i, c\rangle$ → $\langle 0, C(w)\rangle\ \langle 0, C(a)\rangle\ \langle 0, C(b)\rangle$
  - **(2) Fourth symbol** → **third entry** in **dictionary** → $\langle 3, C(a)\rangle$ → **adding pattern** ba as **fourth entry** in **dictionary**
  - **Continuing** in **same fashion** → **building dictionary**

Encoder Output	Dictionary Index	Dictionary Entry
$\langle 0, C(w)\rangle$	01	w
$\langle 0, C(a)\rangle$	02	a
$\langle 0, C(b)\rangle$	03	b
$\langle 3, C(a)\rangle$	04	ba
$\langle 0, C(ƀ)\rangle$	05	$ƀ$
$\langle 1, C(a)\rangle$	06	wa
$\langle 3, C(b)\rangle$	07	bb
$\langle 2, C(ƀ)\rangle$	08	$aƀ$
$\langle 6, C(b)\rangle$	09	wab
$\langle 4, C(ƀ)\rangle$	10	$baƀ$
$\langle 9, C(b)\rangle$	11	$wabb$
$\langle 8, C(w)\rangle$	12	$aƀw$
$\langle 0, C(o)\rangle$	13	o
$\langle 13, C(ƀ)\rangle$	14	$oƀ$
$\langle 1, C(o)\rangle$	15	wo
$\langle 14, C(w)\rangle$	16	$oƀw$
$\langle 13, C(o)\rangle$	17	oo

Development of LZ78 dictionary

Serious LZ78 drawback

- **Growing dictionary** (without bounds)
- **Possible pruning** or **fixed dictionary**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZW

- **LZW** (Lempel-Ziv-Welch, 1984) **approach**
  - **Modification** of **LZ78** by **Terry Archer Welch** (1984)
    - **Removing** necessity of **encoding second element** $\langle i, c \rangle$
    - **Encoder** only sends **index** to **dictionary**
    - **Dictionary** has to be **primed** with **all alphabet letters**
  - **LZW procedure**
    - **Pattern** p in **dictionary** → **index** is **sent**
    - For **added letter** $p * a$ (concatenation) **not** in **dictionary** → **index** for p **sent** → **pattern** $p * a$ **added** to dictionary → **another pattern started** with a
  - **Example**
    - **Sequence** to be **encoded** using **LZW approach** ($\not{b}$ stands for **space**)
      $wabba\not{b}wabba\not{b}wabba\not{b}wabba\not{b}woo\not{b}woo\not{b}woo$
      – **Assuming alphabet** $\{\not{b}, a, b, o, w\}$

Welch, T. A., A Technique for High-Performance Data Compression, Computer, 1984.
Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZW

LZW (Lempel-Ziv-Welch, 1984) approach

Example

- **Sequence** to be **encoded** using **LZW approach** ($\not{b}$ stands for **space**)
  $wabba\not{b}wabba\not{b}wabba\not{b}wabba\not{b}woo\not{b}woo\not{b}woo$
  - **Assuming alphabet** $\{\not{b}, a, b, o, w\}$
- **Encoding procedure**
  - (1) **Encoding** w → **in dictionary** → **concatenate** wa → **not** in **dictionary** → **encoding** w with index $i = 5$ → **adding** wa in **dictionary**
  - (2) **Starting** with a → **in dictionary** → **concatenate** ab → **not** in **dictionary** → **encoding** a with index $i = 2$ → **adding** ab in **dictionary**
  - (3) **Starting** with b → **in dictionary** → **concatenate** bb → **not** in **dictionary** → **encoding** b with index $i = 3$ → **adding** bb in **dictionary**
  - **… repeating steps** until **reaching** w of **second** $wabba$ → **output** of **indices** from **initial dictionary** 5 2 3 3 2 1
  - (4) **Concatenating** wa → **in dictionary** → **concatenate** wab → **not** in **dictionary** → **encoding** wa with index $i = 6$ → **adding** wab in **dictionary** (starting **three-letter entries**)

Index	Entry
1	$\not{b}$
2	a
3	b
4	o
5	w

Index	Entry
01	$\not{b}$
02	a
03	b
04	o
05	w
06	wa
07	ab
08	bb
09	ba
10	$a/\not{b}$
11	$\not{b}$ w
12	$w\ldots$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZW

LZW (Lempel-Ziv-Welch, 1984) approach

Example

- **Sequence** to be **encoded** using **LZW approach** ($\not{b}$ stands for **space**)
  $wabba\not{b}wabba\not{b}wabba\not{b}wabba\not{b}woo\not{b}woo\not{b}woo$
  - **Assuming alphabet** $\{\not{b}, a, b, o, w\}$
- **Encoding procedure**
  - **(4) Concatenating** wa → **in dictionary** → **concatenate** wab → **not** in **dictionary** → **encoding** wa with index $i = 6$ → **adding** wab in **dictionary** (starting **three-letter entries**)
  - **… repeating steps** until **end** of **sequence** → **building dictionary**

Index	Entry	Index	Entry
01	$\not{b}$	14	$a\not{b}\ w$
02	a	15	$wabb$
03	b	16	$ba\not{b}$
04	o	17	$\not{b}\ wa$
05	w	18	abb
06	wa	19	$ba\not{b}\ w$
07	ab	20	wo
08	bb	21	oo
09	ba	22	$o\not{b}$
10	$a\not{b}$	23	$\not{b}\ wo$
11	$\not{b}\ w$	24	$oo\not{b}$
12	wab	25	$\not{b}\ woo$
13	bba		

- **Encoded sequence**
  5 2 3 3 2 1 6 8 10 12 9 11 7 16 5 4 4 11 21 23 4

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZW

LZW (Lempel-Ziv-Welch, 1984) approach

Example

- **Decoding LZW sequence**
  5 2 3 3 2 1 6 8 10 12 9 11 7 16 5 4 4 11 21 23 4
  - **Assuming alphabet** $\{\cancel{b}, a, b, o, w\}$
- **Decoding procedure**
  - **Decoder starts** with **same initial dictionary**
  - **(1) Index** 5 → **corresponds** to w → **decoding** w as **first element** of **decoded sequence** → $\boldsymbol{w}$
  - **(2) Constructing** next **element** in **dictionary** → **pattern** w **exists** in **dictionary** → **nothing** to **add** → $\boldsymbol{w}$
  - **(3) Index** 2 → **corresponds** to a → **concatenate** wa → **not** in **dictionary** → **adding** wa in **dictionary** → starting **new pattern** with a → $\boldsymbol{wa}$
  - … next **four inputs** $3\ 3\ 2\ 1$ → correspond to **letters** $bba\cancel{b}$ → **generating dictionary entries** $ab, bb, ba, a\cancel{b}$ → $\boldsymbol{wabba\cancel{b}}$

Index	Entry
01	$\cancel{b}$
02	a
03	b
04	o
05	w
06	wa
07	ab
08	bb
09	ba
10	$a/\cancel{b}$
11	$\cancel{b}$ w
12	$w\ldots$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: LZW

LZW (Lempel-Ziv-Welch, 1984) approach

Example

- **Decoding LZW sequence**
  5 2 3 3 2 1 6 8 10 12 9 11 7 16 5 4 4 11 21 23 4
  - **Assuming alphabet** $\{\cancel{b}, a, b, o, w\}$
- **Decoding procedure**
  - … next **four inputs** 3 3 2 1 → correspond to **letters** $bba\cancel{b}$ → **generating dictionary entries** $ab, bb, ba, a\cancel{b}$ → $\boldsymbol{wabba\cancel{b}}$
  - (**4**) **Index** 6 → **corresponds** to wa → **decoding** w and a → **concatenate** $\cancel{b}w$ → **not** in **dictionary** → **adding** $\cancel{b}w$ in **dictionary**
  - (**5**) Starting **new pattern** with w → **concatenate** wa → → **pattern** wa → **exists** in **dictionary** → **decode next input**
  - (**6**) **Index** 8 → **corresponds** to bb → **decoding first** b → **concatenate** wab → **not** in **dictionary** → **adding** wab in **dictionary**
  - (**7**) Starting **new pattern** with b → **decoding second** b → → **concatenate** bb → **exists** in **dictionary** → **decode next input**
  - **… repeat steps** and **decode** $wabba\cancel{b}wabba\cancel{b}wabba\cancel{b}wabba\cancel{b}woo\cancel{b}woo\cancel{b}woo$

Index	Entry
01	$\cancel{b}$
02	a
03	b
04	o
05	w
06	wa
07	ab
08	bb
09	ba
10	$a/\cancel{b}$
11	$\cancel{b}$ w
12	$w\ldots$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: Applications

- **Numerous LZxx** and **LZW applications**
  - **LZW** was **initially algorithm** of **choice**
    - **Various applications** → **GIF**, …
  - **Patent concerns** → **increasing use** of **LZ77** algorithm
    - **Popular "deflate" algorithm** → "**zlib**" library, "**gzip**" algorithm, **PNG**, …
- **Graphics Interchange Format (GIF)**
  - **LZW** based **image compression** developed by **CompuServe**
    - **Compressed image stored** with following **parameters**
      - **First byte** → **minimum number** of **bits** b **per pixel** (for example 8 bpp)
      - **Binary** 2^b → "**clear code**" → **resetting compression/decompression** to start-up
      - Initial **dictionary size** 2^{b+1} → when **filled up** → **size doubles** → **until** 4096
      - **LZW codewords** → **stored** as **blocks** of **characters** → **characters** 8 bit long → **maximum block** size 255 → block **terminated** → **terminator eight** 0s
      - **End of image** → end-of-information **code** $2^b + 1$
    - **GIF suitable** for **computer generated** images → **not so** for **natural scenes**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Dictionary techniques: Applications

Portable Network Graphics (PNG)

- **Patent-free replacement** for **GIF** (**LZW** replaced with **LZ77**)
  - **LZ77 compression** based on "**deflate**" **implementation**
    - **Allows** for **match lengths** between 3 and 258
    - Encoder **examines** 3 **bytes** at **each step**
      - If **no match for** at **least** 3 **bytes** → puts out **first byte** → **examines** next 3 **bytes**
    - At **each step** → either **single byte value** or **pair** $\langle \text{match length}, \text{offset} \rangle$ → **combined** to form **alphabet** of **size** 286 (indexed $0 - 285$)
      - **Indices** $0 - 255$ → **value** (literal byte) and 256 → **end-of-block symbol**
      - **Remaining** 29 **indices** $257 - 285$ → **codes** for **ranges** of **lengths between** $3 - 258$
      - **Index values** represented **using Huffman code**

Index	# of Selector Bits	Length	Index	# of Selector Bits	Length	Index	# of Selector Bits	Length
257	0	3	267	1	15,16	277	4	67–82
258	0	4	268	1	17,18	278	4	83–98
259	0	5	269	2	19–22	279	4	099–114
260	0	6	270	2	23–26	280	4	115–130
261	0	7	271	2	27–30	281	5	131–162
262	0	8	272	2	31–34	282	5	163–194
263	0	9	273	3	35–42	283	5	195–226
264	0	10	274	3	43–50	284	5	227–257
265	1	11, 12	275	3	51–58	285	0	258
266	1	13, 14	276	3	59–66			

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Dictionary techniques: Applications

Portable Network Graphics (PNG)

- **Patent-free replacement** for **GIF** (**LZW** replaced with **LZ77**)
  - **LZ77 compression** based on "**deflate**" **implementation**
    - **Allows** for **match lengths** between 3 and 258
    - Encoder **examines** 3 **bytes** at **each step**
      - If **no match for** at **least** 3 **bytes** → puts out **first byte** → **examines** next 3 **bytes**
    - At **each step** → either **single byte value** or **pair** $\langle \text{match length}, \text{offset} \rangle$ → **combined** to form **alphabet** of **size** 286 (indexed $0 - 285$)
      - **Indices** $0 - 255$ → **value** (literal byte) and 256 → **end-of-block symbol**
      - **Remaining** 29 **indices** $257 - 285$ → **codes** for **ranges** of **lengths between** $3 - 258$
      - **Index values** represented **using Huffman code**
    - **Offset values** between $1 - 32\,768$
      - **Divided** into 30 **ranges** and **Huffman encoded**
  - **Estimation** of **pixel values**
    - **Using neighborhood** → **subtracting estimate**
    - **Four estimation models** (upper, left, upper left)
    - **Decorrelation achieved**

Index Ranges	# of Bits	Binary Codes
0–143	8	00110000 through 10111111
144–255	9	110010000 through 111111111
256–279	7	0000000 through 0010111
280–287	8	11000000 through 11000111

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Context based compression: Introduction

- **Techniques** where **minimal prior statistics used**
  - Instead **context** of **data** being **encoded** and **past history**
    - Number of **schemes principally used** for **text compression**
    - Different **ways** on how to use **context** in which **data occurs**
  - **More compression** for **skewed probabilities**
    - **Skewed** probabilities → **certain symbols** with **higher occurrence**
    - Higher **compression performance** for **Huffman/Arithmetic** encoder
    - **Representing symbols** with **greater skew**
      - Utilizing **probability** of **occurrence** in **context** it **occurs**
      - **Examining history** of **sequence** before **determining probabilities**
  - **Example**
    - **English** text with **missing letters** to be **guessed** by subject (Claude Elwood Shannon experiment)

Actual Text	THE ROOM WAS NOT VERY LIGHT A SMALL OBLONG
Subject Performance	____ ROO_____ NOT_ V_____ I_____ SM____ OBL___

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Context based compression: Introduction

- **Techniques** where **minimal prior statistics used**
  - Instead **context** of **data** being **encoded** and **past history**
  - **Example**
    - **English** text with **missing letters** to be **guessed** by subject (Claude Elwood Shannon experiment)

Actual Text	THE ROOM WAS NOT VERY LIGHT A SMALL OBLONG
Subject Performance	____ ROO_____ NOT_ V_____ I_____ SM____ OBL___

      - **Good chance** of **guessing letters** → especially for **ending word letters**
      - **Predicting letters** using **context** → **Shannon** found **upper/lower bounds** of **English alphabet** between 1.3 and 0.6 bits per letter
  - Using **context** in **coder/decoder**
    - Highly **skewed probabilities** → **increasing compression** level
  - **Example**
    - **Encoding** word $(\mathrm{p\ r\ o\ b\ a\ b\ i\ l\ i\ t\ y})$ using **context**
      - Suppose **first** 4 **letters** $(\mathrm{p\ r\ o\ b})$ already **encoded** → **letter** (a) to be **encoded** → **probability ignoring** first **four letters** $p(\mathrm{a}) = 0.06$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Context based compression: Introduction

- **Techniques** where **minimal prior statistics used**
  - Instead **context** of **data** being **encoded** and **past history**
  - **Example**
    - **Encoding** word (p r o b a b i l i t y) using **context**
      - Suppose **first** 4 **letters** (p r o b) already **encoded** → **letter** (a) to be **encoded** → **probability** of (a) **ignoring** first **four letters** $p(\mathrm{a}) = 0.06$
      - Suppose **first** 4 **letters** (p r o b) already **encoded** → **letter** (a) to be **encoded** → **probability** of (a) **considering** previous **letter** (b) **encoded** → **reducing probability** of (q, z, …) → **increasing probability** of (a)
    - Using **more letters** defining **context** (higher-order context)
      - **First order** context (b) for **letter** (a)
      - **Second order** context (ob) for **letter** (a)
      - … **higher order context**
      - **Increasing probability** of **occurrence** (a) → **reducing number** of **bits** for **compression**
    - **Alphabet** of **size** M → **number** of **first-order context** M → **second-order** M^2 → **impractical size** of **distributions**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Context based compression: ppm

- **Techniques** where **minimal prior statistics used**
  - Instead **context** of **data** being **encoded** and **past history**
    - Using **more letters** defining **context** (higher-order context)
      - **Alphabet** of **size** M → **number** of **first-order context** M → **second-order** M^2 → **impractical size** of **distributions**
      - **Encoding sequence** with **alphabet** size 256 using **context order** 5 → 256^5 **probability distributions needed** → 1.09951×10^{12}
- **Prediction with partial match (ppm)**
  - **ppm** is best-known **context-based compression algorithm**
    - **Not** as **popular** as **Ziv-Lempel dictionary** techniques → **lower speed**
    - **Estimating conditional probabilities** as **coding proceeds**
      - **Storing** only **contexts occurring** in **sequence** being **encoded**
      - **Mitigating storage issues** → **not necessary** to **store large number** of **conditional probabilities**
      - **Escape symbol** in **source alphabet** → **signaling letter not seen** in **this context**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Context based compression: ppm

Prediction with partial match (ppm) basic algorithm

- Algorithm **attempts** using **largest context**
  - **Context size predetermined**
  - If **symbol not encountered** previously in **context** → next **smaller context**
  - **Process continues until**
    - **Obtaining context** where **symbol encountered**
    - **Arriving** at **conclusion** where **symbol not encountered** in **any context** → **probability** $1/M$ **used** to **encode symbol** (M alphabet size)
- **Example**
  - **Encoding** letter (a) in **word** (p r o b a b i l i t y) using **context**
    - (1) **First attempt** to see if (p r o b a) **occurred previously** → if (a) **occurred** in **context** (p r o b) → if **not** sending **escape symbol**
    - (2) **Second attempt** to see if (r o b a) **occurred previously** → if (a) **occurred** in **context** (r o b) → if **not** sending **escape symbol**
    - **… continuing** → (a) **occurred** in **zero context** (**occurred** for **first time**) → **probability** to **encode** (a) → **equiprobable model** (-1 context)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Context based compression: ppm

Prediction with partial match (ppm) basic algorithm

- **Variations** of **ppm** algorithm
  - **Numerous variants** related to **assigning counts** to **escape symbol**
    - **Method B** (**ppmb**) → **reducing counts** of **symbols** assigned to **escape**
    - **Method C** (**ppmc**) → **count assigned** to **escape** → **number** of **symbols** occurring in that **context**
- **Length** of **context**
  - **Longer context** → **not** necessarily **better performance**
    - **Longer** maximum **length** → **higher** symbol **probability** with respect to **context**
    - **Longer** maximum **length** → **higher probability** of long **escape sequences**
  - **Compression performance**
    - **Improves** (sharply) with **increasing** maximum **context length** → followed by (steady) **drop** in **performance**
    - **Optimum** for maximum **context length** → **depends** on **source sequence characteristics**
  - **ppm** variant for **deterministic contexts** (**ppm***)

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Context based compression: BWT

Burrows-Wheeler Transform (BWT)

- **Context** used in **different way** for **lossless compression**
  - Major part of **algorithm developed** by **David Wheeler** (1983)
  - **BWT compression** (1994) by **Michael Burrows** and **David Wheeler**
- Requires **entire sequence** available **before coding**
  - **Decoding** procedure **not obvious** from **encoding procedure**
- **BWT encoding algorithm**
  - **Starting** with **sequence** of **length** N
    - (**1**) **Sequence** of **length** $N \rightarrow (N-1)$ **sequences created** with **cyclic shift**
    - (**2**) N **sequences arranged** in **lexicographical order**
    - (**3**) **Encoder** transmits **sequence** of **length** $N \rightarrow$ taking **last letter** of **sorted** and **cyclically shifted** sequences
    - (**4**) Sequence of **last letters** L and **position** of **original sequence** in **sorted list** $\rightarrow$ **coded** and **sent** to **decoder**
  - This **information sufficient** for **recovering original sequence**
    - **Ending** with $(N+1)$ **elements**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Context based compression: BWT

Burrows-Wheeler Transform (BWT)

Example

- **Encoding sequence** (this$\cancel{b}$is$\cancel{b}$the) using **BWT** ($\cancel{b}$ space).
  - (**1**) Performing **cyclic permutations** of **input sequence**
  - (**2**) **Sorting sequences** in **lexicographical order**
  - (**3**) Taking **sequence** L of **last letters**

    L: ssht$\cancel{b}$ii$\cancel{b}$e
  - (**4**) Encoding **sequence** L and **index value** 10

00	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e
01	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t
02	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t	h
03	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t	h	i
04	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t	h	i	s
05	i	s	$\cancel{b}$	t	h	e	t	h	i	s	$\cancel{b}$
06	s	$\cancel{b}$	t	h	e	t	h	i	s	$\cancel{b}$	i
07	$\cancel{b}$	t	h	e	t	h	i	s	$\cancel{b}$	i	s
08	t	h	e	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$
09	h	e	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t
10	e	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h

Permutations of input sequence

- **Sequence** contains **longer runs** of **like letters**

- **Move to front** (**mtf**) algorithm takes **advantage** of **long runs**

0	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t	h	i	s
1	$\cancel{b}$	t	h	e	t	h	i	s	$\cancel{b}$	i	s
2	e	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h
3	h	e	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t
4	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t
5	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t	h
6	i	s	$\cancel{b}$	t	h	e	t	h	i	s	$\cancel{b}$
7	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t	h	i
8	s	$\cancel{b}$	t	h	e	t	h	i	s	$\cancel{b}$	i
9	t	h	e	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$
10	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e

Sequences sorter into lexicographical order

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Context based compression: BWT

Burrows-Wheeler Transform (BWT)

Example

- **Decoding sequence** (L: sshtth$\cancel{b}$iii$\cancel{b}$e) using **BWT** ($\cancel{b}$ space).
  - All **elements** of **initial sequence** (this$\cancel{b}$is$\cancel{b}$the) **included** in (L: sshtth$\cancel{b}$iii$\cancel{b}$e)
  - **Figuring permutations** to **recover original sequence**
  - (**1**) Generating **sequence** F **consisting** of **first element** of **each row** → **simple** due to **lexicographical ordering**
  - (**2**) Sequence F as **lexicographically ordered** (L: sshtth$\cancel{b}$iii$\cancel{b}$e) → F: $\cancel{b}\cancel{b}$ehhiisstt
  - (**3**) Taking (L: sshtth$\cancel{b}$iii$\cancel{b}$e) and (F: $\cancel{b}\cancel{b}$ehhiisstt) to **generate original sequence**
    - **Each row** in table is **cyclical shift** → **first column letter** of **any row** is **letter** appearing **after last column** in **row** in **original sequence**
    - If we know **original sequence** is in kth **row** → unravelling **original sequence starting** with kth **element** of F

0	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t	h	i	s
1	$\cancel{b}$	t	h	e	t	h	i	s	$\cancel{b}$	i	s
2	e	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h
3	h	e	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t
4	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t
5	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t	h
6	i	s	$\cancel{b}$	t	h	e	t	h	i	s	$\cancel{b}$
7	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e	t	h	i
8	s	$\cancel{b}$	t	h	e	t	h	i	s	$\cancel{b}$	i
9	t	h	e	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$
10	t	h	i	s	$\cancel{b}$	i	s	$\cancel{b}$	t	h	e

Sequences sorter into lexicographical order

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

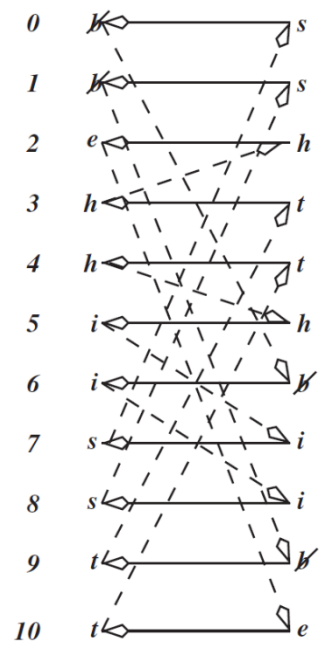
Context based compression: BWT

Burrows-Wheeler Transform (BWT)

Example

- **Decoding sequence** (L: sshtth~~b~~iii~~b~~e) using **BWT** (~~b~~ space).
  - Taking (L: sshtth~~b~~iii~~b~~e) and (F: ~~bb~~ehhiisstt) to **generate original sequence**
  - (1) **Original sequence** number 10
  - (2) **First letter** of **original sequence** $F[10] = t$
  - (3) **Find letter** following (t) in **array** L → **two cases** → **picking lower** $L[4]$ → **next letter reconstructed** $F[4] = h$ → sequence (th …)
  - (4) **Find letter** following (h) in **array** L → **two cases** → **picking lower** $L[6]$ → **next letter reconstructed** $F[6] = i$ → sequence (thi …)
  - … process **continues** getting **original sequence**

$$F = \begin{bmatrix} \not{b} \\ \not{b} \\ e \\ h \\ h \\ i \\ i \\ s \\ s \\ t \\ t \end{bmatrix} \quad L = \begin{bmatrix} s \\ s \\ h \\ t \\ t \\ h \\ \not{b} \\ i \\ i \\ \not{b} \\ e \end{bmatrix}$$



Decoding process

0	$\not{b}$	i	s	$\not{b}$	t	h	e	t	h	i	s
1	$\not{b}$	t	h	e	t	h	i	s	$\not{b}$	i	s
2	e	t	h	i	s	$\not{b}$	i	s	$\not{b}$	t	h
3	h	e	t	h	i	s	$\not{b}$	i	s	$\not{b}$	t
4	h	i	s	$\not{b}$	i	s	$\not{b}$	t	h	e	t
5	i	s	$\not{b}$	i	s	$\not{b}$	t	h	e	t	h
6	i	s	$\not{b}$	t	h	e	t	h	i	s	$\not{b}$
7	s	$\not{b}$	i	s	$\not{b}$	t	h	e	t	h	i
8	s	$\not{b}$	t	h	e	t	h	i	s	$\not{b}$	i
9	t	h	e	t	h	i	s	$\not{b}$	i	s	$\not{b}$
10	t	h	i	s	$\not{b}$	i	s	$\not{b}$	t	h	e

Sequences sorter into lexicographical order

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Context based compression: MTF

- **Burrows-Wheeler Transform (BWT)**
  - **Ending** with sequence of $(N+1)$ **elements** plus **index value**
    - Seems **BWT causes** actual **expansion** not **compression**
    - Transformed **sequence** L can be **compressed more efficiently**
    - **Sequence** L contains **long runs** of **like symbols** → happens **more often** when N **large**
- **Move-to-front (mtf)** coding
  - **Scheme** taking **advantage** of **long runs** of **identical symbols**
    - Starts with **initial listing** of **source alphabet**
    - **Top symbol assigned** number 0, **next symbol** 1, …
    - **First time** symbol **occurs** → corresponding **number transmitted** → **moved** to list **top**
    - **Run** of **identical symbols** → **transformed** to **large number** of 0s
    - **Better results** for **large sized sequences**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Context based compression: MTF

Move-to-front (mtf) coding

Example

- **Encoding sequence** $L = sshtth\cancel{b}iii\cancel{b}e$ using **mtf** ($\cancel{b}$ space), assuming alphabet $\mathcal{A} = \{\cancel{b}, e, h, i, s, t\}$.
  - **(1) Starting** assignment → **first element** in L → (s) → **encoded** as 4 → **move** (s) to **top list**

0	1	2	3	4	5
$\cancel{b}$	e	h	i	s	t

  - **(2) Next element** in L → (s) → **encoded** as 0 → (s) stays **top list**

0	1	2	3	4	5
s	$\cancel{b}$	e	h	i	t

  - **(3) Next element** in L → (h) → **encoded** as 3 → **move** (h) to **top list**

0	1	2	3	4	5
s	$\cancel{b}$	e	h	i	t

  - **(4) Next element** in L → (t) → **encoded** as 5 → **move** (t) to **top list**

0	1	2	3	4	5
h	s	$\cancel{b}$	e	i	t

  - **(5) Next element** in L → (t) → **encoded** as 0 → (t) stays **top list**

0	1	2	3	4	5
t	h	s	$\cancel{b}$	e	i

  - **… continue procedure** getting $4\ 0\ 3\ 5\ 0\ 1\ 3\ 5\ 0\ 1\ 5$

For longer sequences

- **Long runs** of 0s → **encoded** using **run-length encoding (RLE)**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

© Multimedia Technology Group

Context based compression: RLE

Run-length (en)coding (RLE)

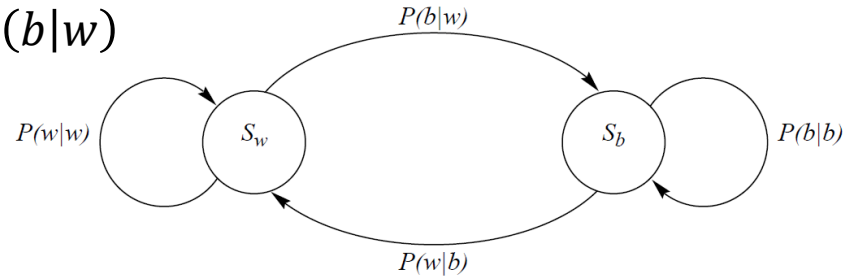
Markov two-state model gives rise to RLE

- Considering **binary image coding** (black and white pixels)
  - **Model** – **probability** of **occurrence** of **white pixel** as the **next observation depends** on whether **current** pixel is **black** or **white**
  - Define **two states** – S_w (current pixel is white), S_b (current pixel is black)
  - **Transition** probability – $P(w|b)$ and $P(b|w)$
  - **State** probabilities – $P(S_w)$ and $P(S_b)$
  - **Definition** of probabilities

    $P(S_w) = 30/31, P(S_b) = 1/31$

    $P(w|w) = 0.99, P(b|w) = 0.01$

    $P(b|b) = 0.70, P(w|b) = 0.30$



  - Using **probability model** and *iid* assumption

    $H = -(30/31)\log(30/31) - (1/31)\log(1/31) = 0.206\text{bit}$
  - Using **Markov model**

    $H(S_b) = -0.30\log 0.30 - 0.70\log 0.70 = 0.881 \text{ bit}$

    $H(S_w) = -0.01\log 0.01 - 0.99\log 0.99 = 0.081 \text{ bit}$

    $H = P(S_b)H(S_b) + P(S_w)H(S_w) = 0.107 \text{ bit}$

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

Context based compression: RLE

Run-length (en)coding (RLE)

Markov two-state model gives rise to RLE

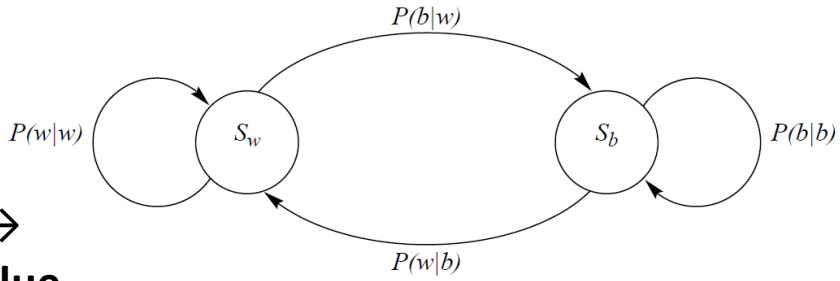
- Considering **binary image coding** (black and white pixels)

- **Highly skewed** nature of **probabilities**
  - **Definition** of probabilities

    $P(S_w) = 30/31, P(S_b) = 1/31$

    $P(w|w) = 0.99, P(b|w) = 0.01$

    $P(b|b) = 0.70, P(w|b) = 0.30$
  - **Once pixel takes value (black/white)** → **highly likely next** pixel having **same value**



Coding length of runs instead of values separately

- For example 190 **white**, 30 **black**, 210 **white pixels** →
  - **Encoding sequence** 190, 30, 210 **indicating value** of **first string** only
  - **Instead** of **coding** 430 **values individually**

Sayood, K.: Introduction to Data Compression, Morgan Kaufmann Publishers, 2006.

References and recommended materials

- [1] Sayood, K.: **Introduction to Data Compression**, Morgan Kaufmann Publishers, 2006.
- [2] Sayood K.: **Lossless Compression Handbook**, Elsevier, 2003.
- [3] Kahrs, M., Brandenburg, K.: **Applications of Digital Signal Processing to Audio and Acoustics**, Kluwer Academic Publishers, 1998.
- [4] Gonzalez, R. C., Woods, R. E.: **Digital Image Processing**, Pearson, 2007.
- [5] Salomon, D., Motta, G.: **Handbook of Data Compression**, Springer-Verlag London, 2010.
- [6] Salomon, D.: **A Concise Introduction to Data Compression**, Springer, 2008.
- [7] Thyagarajan, K. S.: **Still Image and Video Compression with MATLAB**, Willey, 2011.

Thank you for your attention!

MultiMedia Technology Group

Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Radioelectronics

Technická 2
166 27 Prague 6
Czech Republic

© Multimedia Technology Group